

1. Multi-Agent Text2SQL AI Chatbot

1.1 Introduction and Problem Statement

Not very long ago, only a handful of people (Data Engineers, Analysts) could query the SQL databases in an organization. The people who needed it the most—Sales and Marketing teams—didn't have the skills to query the databases. They had to rely on the technical teams to get information.

This multi-agent Text2SQL chatbot removes that dependency. It empowers all people in the organization to ask questions in natural English language.

The Challenge:

- Technical Barrier: Non-technical stakeholders cannot query databases directly
- SQL Complexity: Writing complex JOINS and aggregations requires expertise
- Time Consumption: Back-and-forth with data teams slows decision-making
- Visualization Gap: Raw query results don't provide immediate insights
- Error Prone: Manual SQL queries are susceptible to syntax and logical errors

The Solution: A conversational AI system that:

- Accepts natural language questions in plain English
- Validates scope to prevent irrelevant queries
- Generates optimized SQL queries automatically
- Handles errors intelligently with retry mechanisms
- Provides natural language explanations of results
- Creates visualizations when beneficial
- Streams execution process for transparency

1.2 Tech Stack for AI Component

- **Core Framework:** LangGraph (state machine architecture)
- **Frontend:** Angular (chat interface)
- **AI s LLM:** OpenAI API (gpt-4o-mini) or alternative LLM providers (Anthropic Claude, Ollama for local deployment)
- **Visualization:** Plotly (LLM-Generated Visualization Code)
- **Utilities:** python-dotenv, Pandas

1.3 Multi-Agent Architecture

The chatbot uses a multi-agent architecture where specialized AI agents work together to provide accurate, context-aware responses.

Agent Definitions:

Agent	Role	Responsibility
Guardrails Agent	Security and Scope Manager	Filters questions to ensure they are relevant to e-commerce data analysis, identifies greetings, and blocks out-of-scope queries
SQL Agent	SQL Expert	Generates only valid SQL queries without any formatting or explanation based on the database schema
Analysis Agent	Data Analyst	Explains database query results in natural language with clear insights
Visualization Agent	Visualization Specialist	Generates clean, executable Plotly code for data visualization when beneficial
Error Agent	Error Recovery Specialist	Diagnoses and fixes SQL errors with expert knowledge of database schemas

1.4 Agentic Flow

The system follows this workflow:

1. **User Input:** User asks a question in natural language
2. **Guardrails Check:** Guardrails Agent validates if the question is in scope
 - If greeting → sends friendly welcome message and ends
 - If out-of-scope → returns predefined rejection message
 - If in-scope → continues to SQL generation

3. **SQL Generation:** SQL Agent converts natural language into valid SQL query
4. **Query Execution:** System executes SQL safely against the database
5. **Error Handling:** If error occurs, Error Agent diagnoses and attempts to fix
6. **Analysis:** Analysis Agent explains results in natural language
7. **Visualization:** If beneficial, Visualization Agent generates Plotly charts
8. **Response:** Final answer with explanation and optional visualization returned to user

1.5 Agent State Management

Using TypedDict for state management ensures type safety and provides benefits:

- Python will warn if wrong data types are used
- Editor provides autocomplete and reduces mistakes
- State includes: question, sql_query, query_result, error, final_answer, visualization_code, is_in_scope, iteration_count

1.6 Agent Configurations Example

```
AGENT_CONFIGS = {
    "guardrails_agent": {
        "role": "Security and Scope Manager",
        "system_prompt": "You are a strict guardrails system that filters
questions
                        to ensure they are relevant to e-commerce data analysis."
    },
    "sql_agent": {
        "role": "SQL Expert",
        "system_prompt": "You are a senior SQL developer specializing in e-
commerce
                        databases. Generate only valid SQL queries without any
                        formatting or explanation."
    },
    "analysis_agent": {
        "role": "Data Analyst",
        "system_prompt": "You are a helpful data analyst that explains database
                        query results in natural language with clear insights."
    },
    "viz_agent": {
        "role": "Visualization Specialist",
        "system_prompt": "You are a data visualization expert. Generate clean,
                        executable Plotly code without markdown formatting."
    },
    "error_agent": {
        "role": "Error Recovery Specialist",
        "system_prompt": "You diagnose and fix SQL errors with expert knowledge
                        of database schemas and query optimization."
    }
}
```

1.7 Example Interactions

User Query (Natural Language)	Expected Chatbot Response
"Show me sales by category for last month"	Generate bar chart showing category sales with date filter applied
"What are my top 5 customers by revenue?"	Display ranked table or horizontal bar chart of top customers
"Compare this month vs last month"	Show comparison chart with period-over-period metrics
"Which products have the lowest ratings?"	Return sorted list of products by average rating
"What's the trend in order cancellations?"	Generate time series line chart of cancellation rates
"How many orders were shipped by air?"	Return count with natural language explanation

1.8 Role-Based Chatbot Access

The chatbot must respect data segregation principles based on user roles:

Role	Data Access Scope
------	-------------------

Individual	Own orders, purchases, reviews, and spending patterns only
-------------------	--

Corporate	Own store's products, orders, customers, reviews, and sales data
------------------	--

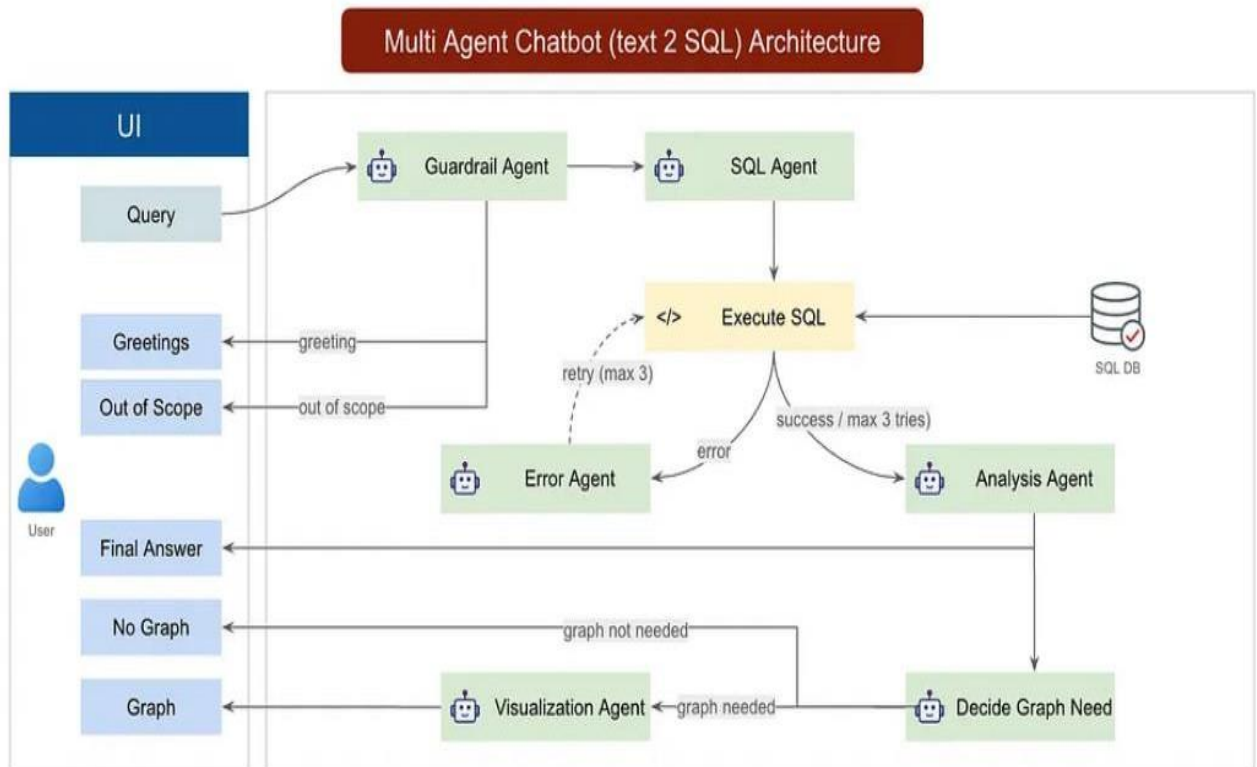
Admin	Full platform access including all stores, users, and aggregate data
--------------	--

5.G Integration with Spring Boot

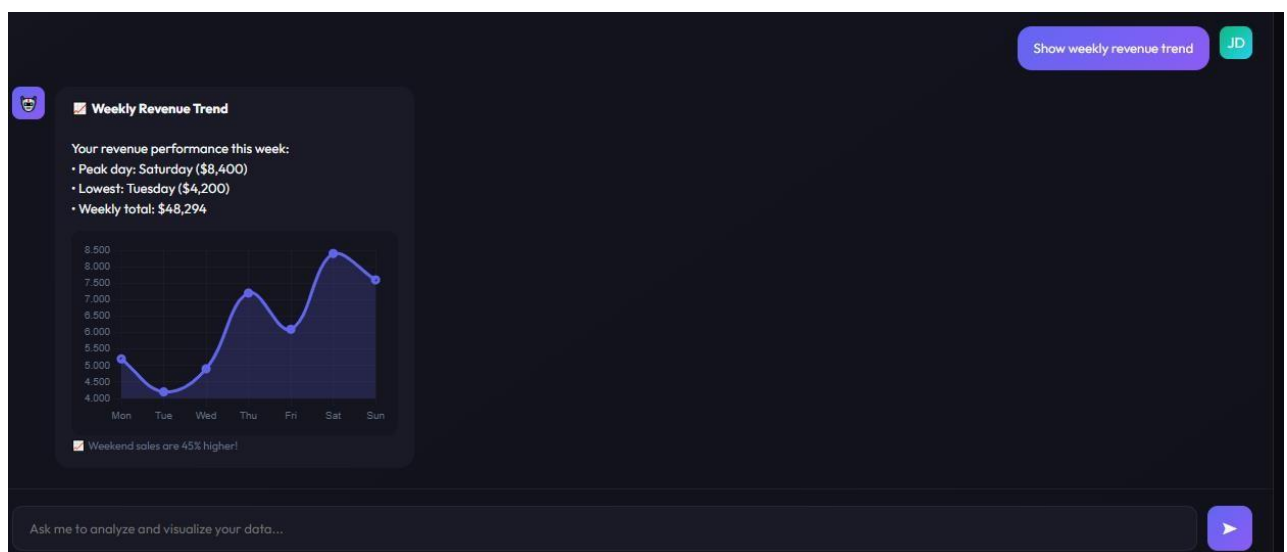
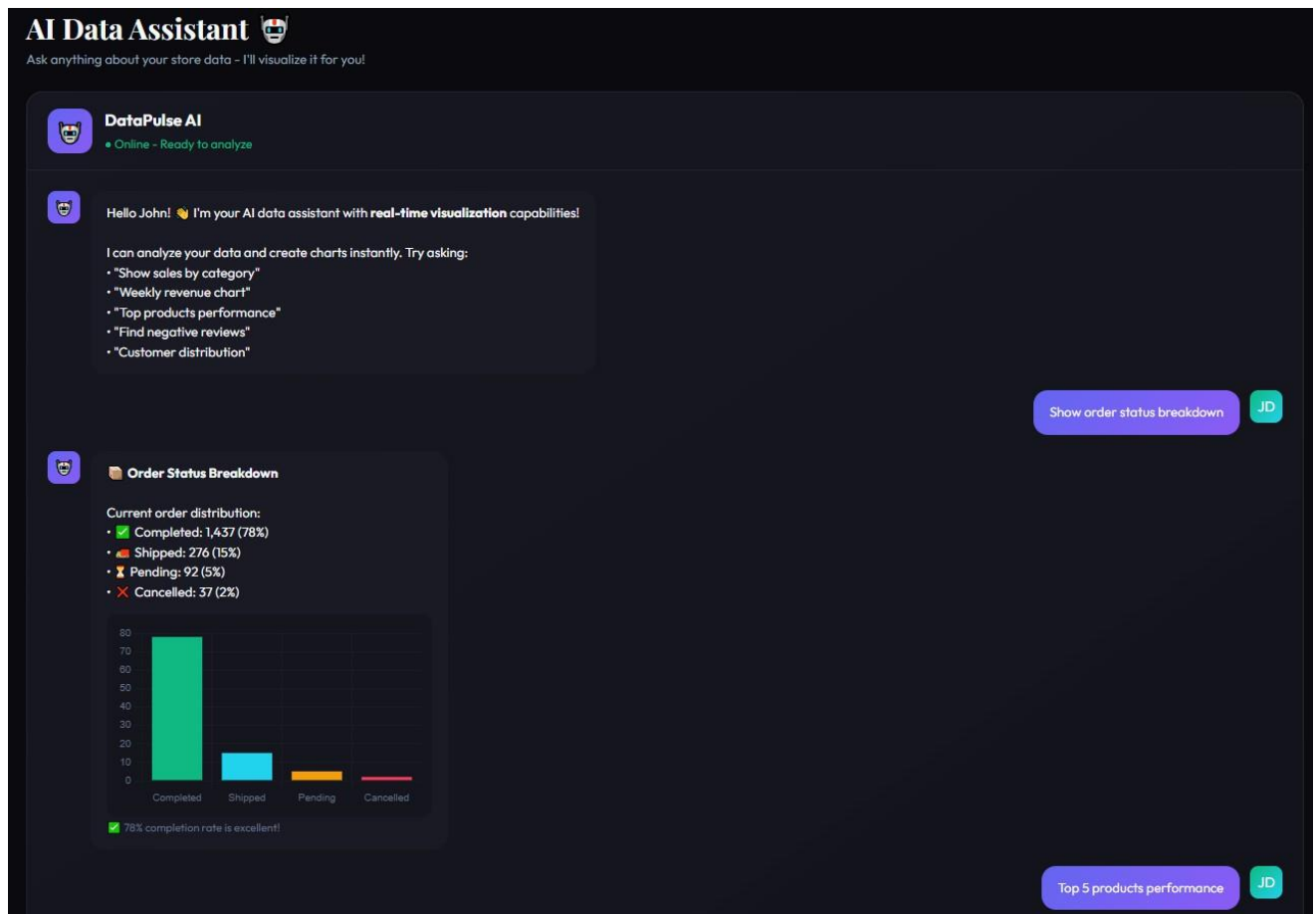
The AI chatbot service (Python/LangGraph) communicates with the Spring Boot backend via REST API or message queue. Students should implement:

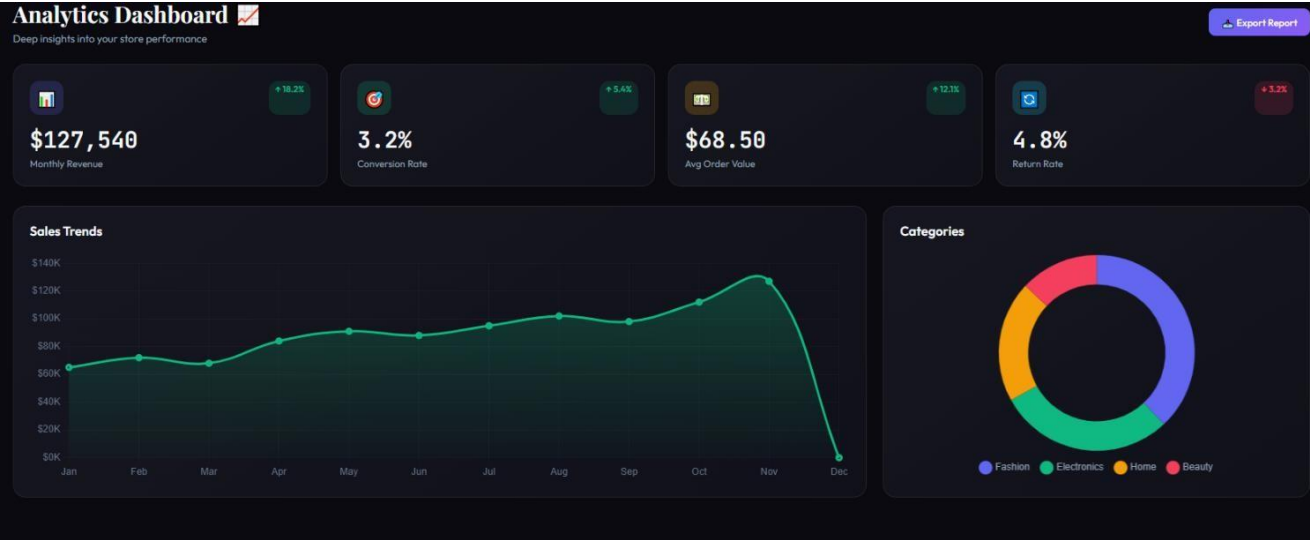
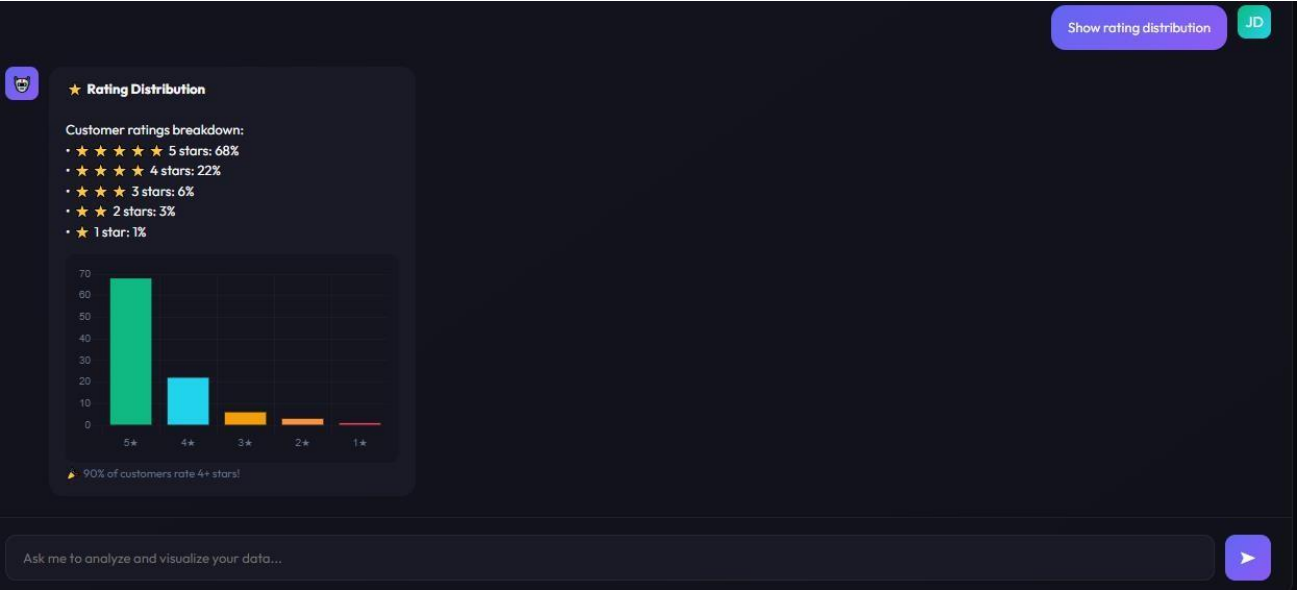
- Dedicated endpoint for chat requests (/api/chat/ask)
- Query execution service that runs generated SQL safely
- Result formatting and passing to visualization agent
- Session management for conversation context

Multi Agent Chatbot Architecture



Example Screen





Orders Management

Track and manage all orders

New Order

All Status

All Time

ORDER ID	CUSTOMER	PRODUCTS	TOTAL	DATE	STATUS
#ORD-7821	SM Sarah Miller	3 Items	\$284.00	Dec 2, 2024	Completed
#ORD-7828	JW James Wilson	2 Items	\$156.50	Dec 2, 2024	Shipped
#ORD-7819	EJ Emily Johnson	5 Items	\$432.00	Dec 1, 2024	Pending
#ORD-7818	MB Mike Brown	1 Item	\$89.99	Dec 1, 2024	Cancelled

Products Catalog

Manage your inventory

+ Add Product

All Categories

Search products...



Classic Cotton T-Shirt

\$29.99

124 in stock



Running Sneakers Pro

\$89.99

56 in stock



Wireless Headphones

\$149.99

32 in stock



Smart Watch Series X

\$299.99

18 in stock



Leather Handbag

\$199.99

45 in stock



Designer Sunglasses

\$129.99

67 in stock



Premium Lipstick Set

\$49.99

89 in stock



Decorative Lamp

\$79.99

34 in stock

Customer Management

View and manage your customers

+ Add Customer



+12.4%

3,421

Total Customers



+28.3%

284

New This Month



+5.2%

892

Gold Members



+15.8%

\$142

Avg. LTV

CUSTOMER	MEMBERSHIP	TOTAL SPEND	ORDERS	STATUS
 Sarah Miller New York	 Gold	\$1,284.88	14	 Active
 James Wilson Los Angeles	 Silver	\$756.58	8	 Active
 Emily Johnson Chicago	 Bronze	\$432.88	5	 Active

Shipments Tracking

Monitor all shipments in real-time

+ Create Shipment



156

Pending



423

In Transit



1,247

Delivered



23

Returns

TRACKING ID	ORDER	CUSTOMER	CARRIER	DESTINATION	STATUS	ETA
TRK-892341	#7820	James Wilson	FedEx	Los Angeles, CA	 In Transit	Dec 4
TRK-892348	#7819	Emily Johnson	UPS	Chicago, IL	 Processing	Dec 5
TRK-892339	#7815	Michael Lee	DHL	Miami, FL	 Delivered	Dec 1